



IceWeatherstation

Stephans Version – Schritt-für-Schritt inkl. Verkabelung

Reduzierte Variante: BME280, OLED-Display, wasserdichter Temperaturfühler,
Windrichtung, Windgeschwindigkeit, Regenmenge, Luftqualität (MQ135), Betriebs-LED

Ohne Blitzsensor und ohne Schallpegelsensor

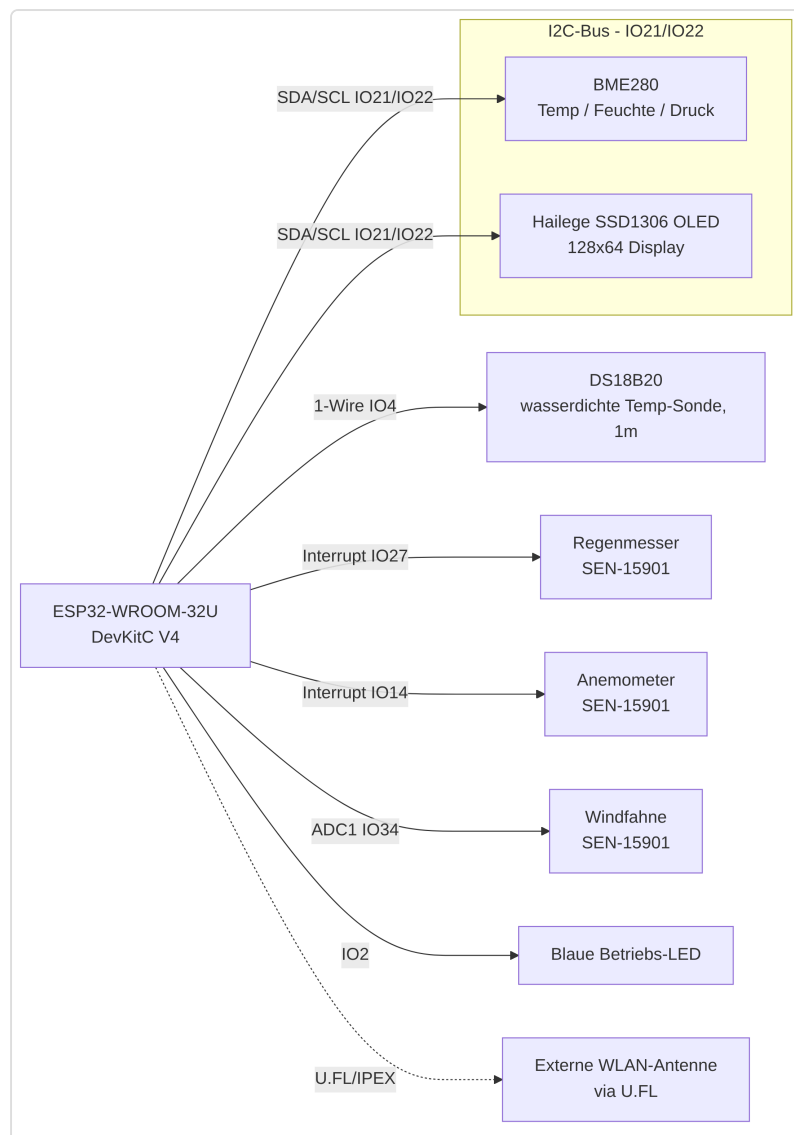
Erstellt: 08.08.2026 · Pin-Bezeichnungen wie auf dem ESP32-Board aufgedruckt
Vollständige Projektdoku (Originalgerät, mehr Sensoren): github.com/icepaule/IceWeatherstation

Inhalt

1. Überblick — was dein Gerät kann
2. Was schon erledigt ist
3. Teileliste
4. Verkabelung + Pin-Tabelle (Board-Beschriftung)
5. Sensoren einzeln anschließen und testen
6. Eigenes WLAN eintragen
7. Status-LED
8. Gehäuse- und Mastmontage
9. Kalibrierung nach dem Aufbau
10. Fertig — Werte ablesen
11. Troubleshooting

1. Überblick — was dein Gerät kann

Dein Gerät ist eine reduzierte Variante der IceWeatherstation: dieselbe Basis (ESP32 + Tasmota-Firmware), aber mit weniger Sensoren als das Originalgerät. Kein Blitzsensor, kein Schallpegelsensor — dafür einfacher im Aufbau. Alle Werte erscheinen automatisch auf einer eigenen Webseite im Browser (kein Cloud-Konto, keine App nötig) und zusätzlich live auf dem kleinen OLED-Display am Gehäuse.



Blockschaltbild deiner Version: BME280 und OLED teilen sich einen I2C-Bus, alle anderen Sensoren hängen einzeln am ESP32.

Was das Gerät misst

Messgröße	Sensor
Temperatur, Luftfeuchte, Luftdruck	BME280
Zusatztemperatur (wasserdicht, außen)	DS18B20, 1m Kabel
Windgeschwindigkeit	SparkFun SEN-15901 (Anemometer)
Windrichtung	SparkFun SEN-15901 (Windfahne)
Niederschlagsmenge	SparkFun SEN-15901 (Regenmesser/Kippwaage)
Anzeige direkt am Gehäuse	Hailege 0,96" OLED, 128×64, I2C
Luftqualität (CO2-ähnlich, intern)	MQ135-Gassensor, im geschlossenen Hauptgehäuse
Betriebsanzeige	Blaue Status-LED (blinkt bei Web-UI-Zugriff, siehe Abschnitt 7)

2. Was schon erledigt ist

Das Board ist bereits fertig vorkonfiguriert — die Firmware wurde geflasht, alle GPIOs für genau deine Sensoren gesetzt, das OLED-Display softwareseitig eingerichtet und das passende Dashboard-Skript hochgeladen. Das wurde am 08.08.2026 zum Testen kurz mit einem eigenen (privaten) WLAN verbunden — das trägst du in Schritt 6 auf dein eigenes WLAN um, das ist der einzige Software-Schritt, der noch nötig ist.

Konkret schon erledigt:

- Firmware geflasht: Tasmota 15.5.0 (Sonder-Build mit OLED-Unterstützung)
- GeräteName gesetzt: `Wetterstation Stephan`
- Alle GPIOs für deine Sensoren konfiguriert (Tabelle in Abschnitt 4)
- OLED-Display softwareseitig aktiviert (uDisplay/SSD1306)
- Dashboard-Skript hochgeladen (zeigt WLAN/Wind/Regen/Temperatur/Druck/Feuchte auf dem Display)
- Zeitzone (Europe/Berlin, Sommerzeit) gesetzt
- MQ135-Luftqualitätssensor per Eigenkalibrierung eingebunden (Details Abschnitt 5.7)
- Temperatur gegen Referenzthermometer kalibriert (`TempOffset`)
- Status-LED blinkt bei Web-UI-Zugriff (Details Abschnitt 7)
- Kein MQTT/Home Assistant konfiguriert — bewusst, dein Gerät braucht das nicht, alles läuft übers Web-UI

Was du noch tun musst: Sensoren verkabeln (Abschnitt 4+5), eigenes WLAN eintragen (Abschnitt 6), Gehäuse/Mast montieren (Abschnitt 8), Windfahne kalibrieren (Abschnitt 9).

3. Teileliste

#	Bauteil	Hinweis
1	ESP32-WROOM-32U DevKitC V4 (extern Antenne)	bereits geflasht/konfiguriert — das Board, das du hast
2	WLAN-Antenne, 2,4 GHz, U.FL/IPEX	nicht direkt an Metallteilen montieren
3	Bosch BME280 I2C-Breakout	SDA/SCL gemeinsam mit OLED
4	Hailege 0,96" I2C OLED-Display, 128×64, 4-Pin	gemeinsamer I2C-Bus mit BME280
5	DS18B20, wasserdicht, 1m Kabel	+ 4,7 kΩ Pull-up-Widerstand nötig
6	SparkFun Weather Meter Kit SEN-15901	Regenmesser + Anemometer + Windfahne in einem Set
7	Blaue LED (Betriebsanzeige) + 220Ω-Vorwiderstand	einfache Einzel-LED reicht

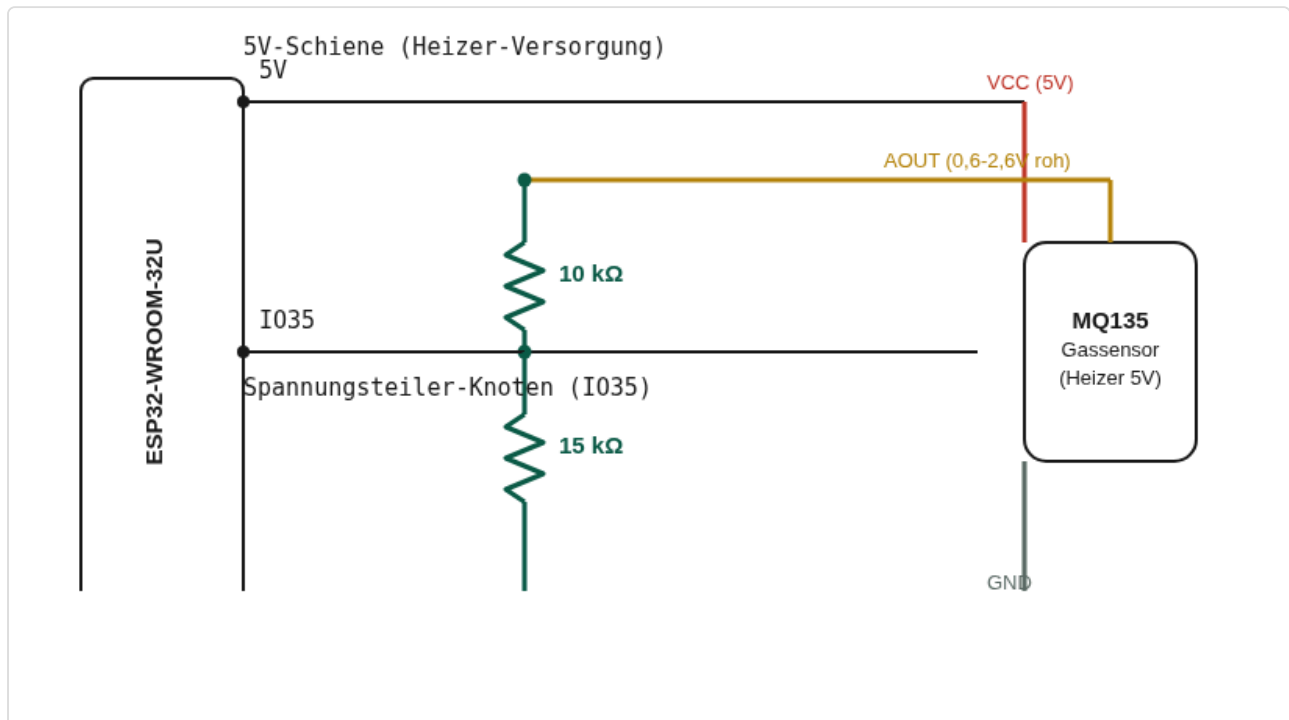
7b	MQ135-Gassensor (4-Pin-Breakout)	Heizer braucht 5V, im geschlossenen Hauptgehäuse montiert (kein Außenluftkontakt)
8	Widerstände: 1× 4,7 kΩ (DS18B20-Pullup), 1× 10 kΩ (Windfahnen-Spannungsteiler), 1× 10 kΩ + 1× 15 kΩ (MQ135-Spannungsteiler)	1%-Toleranz empfohlen
9	Elektronik-Gehäuse, IP65, ~200×120×75mm	für ESP32 + BME280 + Klemmleisten
10	Kabelverschraubungen IP68, gemischte Größen	Wetterkabel-Bündel, DS18B20-Sonde, Stromkabel
11	Mast/Rohrhalterung, Aluminium, U-Bolzen	Anemometer/Windfahne brauchen freien Luftstrom
12	Wetterfeste 5V/USB-Stromzuleitung	einfaches "dummes" Netzteil, kein Schnelllader (siehe Warnhinweis Abschnitt 8)
13	Schraub-/Verbindungsklemmen, Dupont-Kabel	Klemmen statt Löten = leichtere Fehlersuche
14	Silikagel-Beutel	gegen Kondenswasser im Gehäuse

4. Verkabelung + Pin-Tabelle (Board-Beschriftung)

Die Pins sind hier mit der Beschriftung angegeben, die direkt auf deinem ESP32-Board neben den Stiftleisten aufgedruckt ist (z.B. "IO21") — du musst also nicht zwischen GPIO-Nummern und Board umrechnen.

Bauteil	Pin/Ader	Board-Pin	Hinweis
BME280	VCC	3V3	
BME280	GND	GND	
BME280	SDA	IO21	gemeinsamer I2C-Bus mit OLED
BME280	SCL	IO22	gemeinsamer I2C-Bus mit OLED
Hailege OLED	VCC	3V3	
Hailege OLED	GND	GND	
Hailege OLED	SDA	IO21	dritter Teilnehmer am selben I2C-Bus
Hailege OLED	SCL	IO22	dritter Teilnehmer am selben I2C-Bus
DS18B20	VCC (meist rot)	3V3	
DS18B20	GND (meist schwarz)	GND	
DS18B20	DATA (meist gelb)	IO4	+ 4,7 kΩ Pull-up zwischen DATA und 3V3 (zwingend!)
Blaue Status-LED	Anode (langes Beinchen) über 220Ω-Vorwiderstand	IO2	
Blaue Status-LED	Kathode (kurzes Beinchen)	GND	
SEN-15901 Regenmesser	Ader A	GND	Reed-Kontakt, polaritätsfrei
SEN-15901 Regenmesser	Ader B	IO27	
SEN-15901 Anemometer	Ader A	GND	Reed-Kontakt, polaritätsfrei
SEN-15901 Anemometer	Ader B	IO14	
SEN-15901 Windfahne	Ader A	GND	variabler Widerstand, polaritätsfrei

SEN-15901 Windfahne	Ader B	Spannungsteiler- Knoten → IO34	+ fester 10 kΩ-Widerstand zwischen 3V3 und diesem Knoten
MQ135	VCC (Heizer)	5V (nicht 3V3!)	Heizer zieht durchgehend ~150– 200 mA
MQ135	GND	GND	
MQ135	AOUT	Spannungsteiler- Knoten → IO35	+ 10 kΩ (AOUT → Knoten) und 15 kΩ (Knoten → GND), siehe Diagramm unten



MQ135-Spannungsteiler: 10 kΩ zwischen AOUT und dem IO35-Messknoten, 15 kΩ zwischen diesem Knoten und GND
— reduziert den 0–5V-Rohausgang auf den 0–3,3V-Bereich, den der ESP32-ADC verträgt.

Software-Konfiguration (bereits erledigt, nur zur Info): Jeder Pin oben wurde intern folgender GPIO-Rolle zugewiesen — falls du das Board mal neu flashen musst, hier der komplette Befehl:

```
Backlog GPIO21 640;GPIO22 608;GPIO4 1312;GPIO27 352;GPIO14 353;GPIO34
4704;GPIO35 4864;GPIO32 6210;GPIO2 288
```

(Stand 08.08.2026: IO35 nutzt die Rolle "ADC Range1" statt "ADC Input1" — ein ESP32 unterstützt nur eine Instanz von "ADC Input" gleichzeitig, die belegt die Windfahne (IO34). IO35 zusätzlich mit `AdcParam2 6,0,4095,0,4095` auf reine 1:1-Durchreichung gestellt (die Rolle "ADC Range" würde sonst eine eigene Skalierung anwenden) — dadurch zeigt der MQ135-Rohwert im Web-UI leider unter der fest einkompilierten Fehlbezeichnung "Schallpegel...dBA" (Custom-Firmware-Label-Patch vom Originalgerät, dort war "Range" tatsächlich der dBA-Sensor; nur per Firmware-Neubau behebbar). Die echte, korrekt benannte "Luftqualität"-Zeile im Web-UI ist die maßgebliche Anzeige. IO2 nutzt die Rolle "Led" statt "LedLink" — "LedLink" hat ein eigenes, nicht abschaltbares Verbindungsstatus-Verhalten, das eine frei steuerbare Anzeige-LED stört.) (IO32 ist ein rein virtueller Marker fürs OLED, kein physischer Sensor-Pin — nicht verkabeln.)

RJ11-Pinbelegung SEN-15901

Sensor	Kabel	Belegte Pins	Funktion
Regenmesser	eigenes, separates Kabel	die beiden mittleren Kontakte	Reed-Kontakt (Schalter)
Anemometer	gemeinsames Kabel mit Windfahne	Pins 2+3 (innere Adern)	Reed-Kontakt (Schalter)
Windfahne	gemeinsames Kabel mit Anemometer	Pins 1+4 (äußere Adern)	8-Stufen-Widerstandsnetzwerk, 891 Ω –120 k Ω

Aderfarben sind nicht herstellerübergreifend einheitlich — im Zweifel per Multimeter gegen die Pin-Nummern verifizieren, bevor final verlötet wird.

Kein Spannungsteiler zur Stromversorgung! Ein Widerstandsteiler funktioniert nur für Signale (wie bei der Windfahne), niemals um den ESP32 selbst zu versorgen — der zieht schwankenden Strom (WLAN-Spitzen bis 300–500 mA), ein Teiler würde einbrechen (Reset-Schleifen). 5V-Zuleitung einfach direkt an den 5V-Pin (Board hat einen eingebauten LDO-Regler auf 3,3V).

5. Sensoren einzeln anschließen und testen

Nacheinander anschließen und nach jedem Sensor sofort prüfen — das spart bei einem Fehler das Durchsuchen mehrerer gleichzeitig neuer Kabel. Öffne dazu im Browser die Adresse des Geräts (siehe Abschnitt 6) → *Konsole*, und gib die Befehle unten dort ein.

5.1 DS18B20

VCC/GND/DATA anschließen + **zwingend 4,7 kΩ Pull-up zwischen DATA und 3V3** (ohne Pull-up bleibt der Sensor unsichtbar — kein Fehler, sondern erwartetes Verhalten eines offenen 1-Wire-Busses). Prüfen: `Status 10` → `DS18B20:{"Temperature":...}` sollte erscheinen.

5.2 Windfahne

Spannungsteiler mit festem 10 kΩ-Widerstand zwischen 3V3 und IO34, Windfahne selbst zwischen IO34-Knoten und GND. Prüfen: `Status 10` → `ANALOG:{"A1":...}`.

Live gefunden (08.08.2026): Ein loser Kontakt am Spannungsteiler-Knoten (10-kΩ-Widerstand oder Windfahnen-Ader) zeigt sich als **ständig schwankender A1**-Wert, obwohl die Fahne stillsteht (z.B. 188 → 245 → 184 ohne Muster). Fix: alle drei Verbindungen am Knoten (Widerstand-Beinchen ×2, Windfahnen-Ader) fest nachdrücken/nachlöten — danach steht der Wert bei konstanter Ausrichtung felsenfest.

5.3 Regenschmesser + Anemometer

`CounterDebounce` ist bereits auf `10` gesetzt (Entprellung). Test: Reed-Kontakte per Hand kurz schließen bzw. Anemometer drehen — `COUNTER:{"C1":..., "C2":...}` sollte hochzählen (C1 = Regen, C2 = Wind).

5.4 BME280

Gemeinsamer I2C-Bus mit dem OLED. `I2CScan` sollte `0x76` (oder `0x77`) finden.

5.5 OLED-Display

Ist bereits softwareseitig eingerichtet (uDisplay, `DisplayModel 17`). Nach dem Anschließen an IO21/IO22/3V3/GND sollte es automatisch die vier Dashboard-Zeilen zeigen. Falls nicht: Boot-Log prüfen, dort sollte `DSP: SSD1306 initialized` stehen — falls dein Display auf Adresse `0x3D` statt `0x3C` läuft, sag Bescheid, dann muss der Descriptor einmal angepasst werden.

5.6 Blaue Status-LED

Über 220Ω-Vorwiderstand an IO2 (Anode) und GND (Kathode). Details zum Blink-Verhalten: Abschnitt 7.

5.7 MQ135 (Luftqualität)

Spannungsteiler wie im Diagramm in Abschnitt 4 (10 kΩ zwischen AOUT und dem IO35-Knoten, 15 kΩ zwischen diesem Knoten und GND), Heizer an **5V** (nicht 3V3). Prüfen: `Status 10` → `ANALOG:{"Range1":...}` sollte einen Rohwert zwischen 0 und 4095 zeigen.

Der Sensor ist bereits per Eigenkalibrierung eingebunden (kein Standard-Tasmota-Feature, siehe Kasten unten) und braucht nach jedem Neustart **180 Sekunden in normaler/sauberer Luft**, bevor die Web-UI-Zeile "Luftqualität" einen echten Wert statt "Kalibriere.." zeigt. Neustart also nicht direkt nach Kochen/Rauchen/Lüften auslösen, sonst verzerrt das die Referenz bis zum nächsten Neustart.

Warum eine Eigenkalibrierung nötig war: Tasmotas eingebaute MQ-Sensor-Formel (Kommando `AdcParam` Typ 10) geht von einem anderen Spannungsteiler-Aufbau aus als unserem und lieferte am realen Sensor dauerhaft unplausible Werte, unabhängig von der Laufzeit. Das Dashboard-Skript berechnet den ppm-Wert deshalb komplett selbst aus dem rohen ADC-Wert (Formel und Kalibrier-Konstanten identisch zu bereits seit Tagen produktiv laufenden, baugleichen MQ135-Sensoren im eigenen Heimnetz, nur mit unserem eigenen Spannungsteiler-Verhältnis).

6. Eigenes WLAN eintragen

Das Gerät wird dir bereits im Ersteinrichtungs-Zustand übergeben — es öffnet direkt nach dem Einschalten seinen eigenen WLAN-Zugangspunkt (Name beginnt mit `tasmota-`) mit einer Auswahlseite für dein WLAN, genau wie bei einem neuen Router.

1. Mit dem Handy/Laptop mit diesem WLAN verbinden
2. Es sollte sich automatisch eine Setup-Seite öffnen (falls nicht: im Browser `192.168.4.1` aufrufen)
3. Dein eigenes WLAN aus der Liste wählen, Passwort eintragen, speichern

Danach ist das Gerät in deinem eigenen WLAN erreichbar — der Gerätenamen (`tasmota-XXXXXX`) bleibt gleich, nur das Netzwerk wechselt. Als Lesezeichen speichern, das ist später deine "App". Falls der mDNS-Name (`.local`) bei dir nicht auflöst, zeigt dein Router (DHCP-Client-Liste) die aktuelle IP-Adresse unter dem Gerätenamen `Wetterstation Stephan`.

Alternative über die Konsole, falls die Setup-Seite mal nicht erscheint: `Backlog SSID1 <DEIN_WLAN_NAME>;Password1 <DEIN_WLAN_PASSWORT>`

7. Status-LED

Deine LED blinkt bei **WLAN-Datentransfer** — jedes Mal, wenn die Web-UI-Seite offen ist und sich aktualisiert, blitzt sie kurz auf (ca. 400ms), aber höchstens einmal alle 2 Sekunden. Anders als beim Originalgerät (das MQTT nutzt und dort `LedState 7` für "blinkt bei jeder MQTT-Aktivität" verwendet) hat Tasmota für generischen WLAN-Traffic keinen eingebauten Blink-Modus — dein Gerät braucht ja bewusst kein MQTT. Das Dashboard-Skript steuert die LED deshalb komplett manuell (`LedState 0` + gezielte `LedPower1` -Befehle).

In der Nachtruhe (22:00–08:00 Uhr) bleibt die LED durchgehend aus, damit sie nachts niemanden stört.

Wichtig für die GPIO-Rolle: IO2 muss die Rolle **"Led"** haben, nicht "LedLink" — Letztere hat ein eigenes, fest verdrahtetes Verbindungsstatus-Verhalten (springt selbstständig auf `LedState 8` , "an wenn WLAN verbunden") und überschreibt jede manuelle Steuerung. Live gefunden und korrigiert (08.08.2026) — falls die LED bei dir dauerhaft leuchtet statt zu blinken, `GPIO2` prüfen (Konsole: `GPIO2`) — muss `{"Led1":288}` zeigen, nicht `{"LedLink":544}` .

Wichtig: dein Browser-Tab muss aktiv/fokussiert sein — die meisten Browser drosseln JavaScript-Timer in Hintergrund-Tabs stark, dann wirkt es so, als würde die LED gar nicht blinken, obwohl der Mechanismus läuft. Ein Reload mit aktivem Tab behebt das.

8. Gehäuse- und Mastmontage

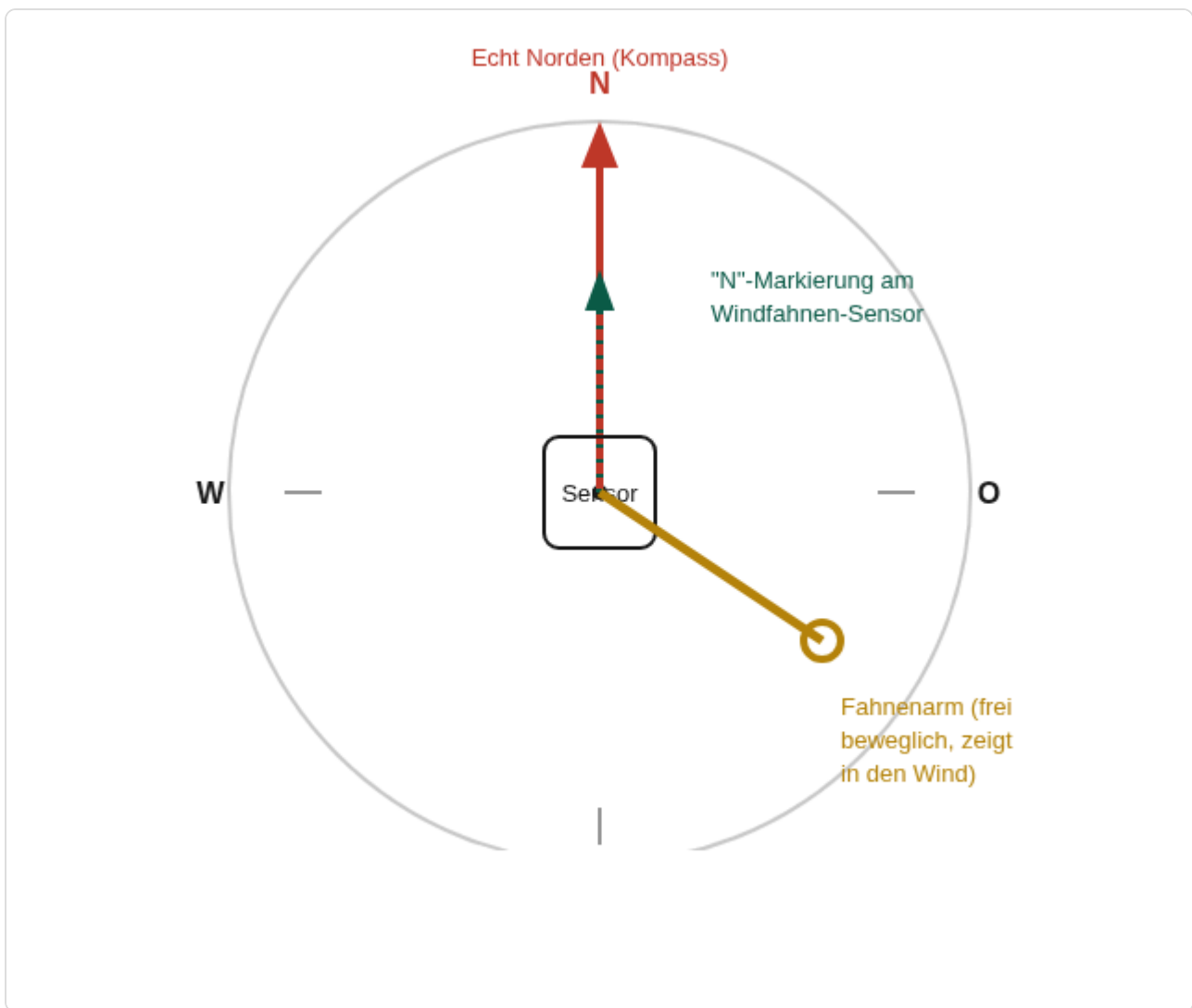
1. Alle Sensoren final ins IP65-Gehäuse verkabeln (Zugentlastung an jeder Kabelverschraubung), Schraubklemmen statt Direktverlötung (leichtere Fehlersuche später)
2. Silikagel-Beutel ins Gehäuse legen
3. Anemometer/Windfahne so hoch und frei wie möglich montieren, Regenschirm exakt waagrecht (Wasserwaage!)
4. Vor dem endgültigen Verschließen nochmal **Status 10** prüfen — sind alle Werte nach dem finalen Verkabeln noch plausibel?

Stromversorgung — zwei Fallstricke aus dem Originalaufbau:

- **Kein Spannungsteiler zur Versorgung!** 5V-Pin direkt anschließen (eingebauter LDO-Regler).
- **Keine USB-C-Schnelllade-Netzteile (PD/QC)!** Manche liefern ganz ohne Aushandlung mit einer "intelligenten" Gegenstelle keine Spannung — das Board bleibt dann komplett dunkel. Einfaches "dummes" 5V/1-2A-USB-Netzteil verwenden (z.B. altes Handy-Ladegerät).

9. Kalibrierung nach dem Aufbau

Windfahne: Die Tabelle im Dashboard-Skript ist bereits aus dem offiziellen Hersteller-Datenblatt der SEN-15901-Sensorik berechnet (Widerstand → Richtung, umgerechnet auf unseren 10-kΩ-Spannungsteiler) — **kein Software-Abgleich mehr nötig**. Die Kalibrierung reduziert sich auf einen rein mechanischen Schritt bei der Mastmontage:



1. Windfahne auf ihre Achse/Lager montieren (noch nicht endgültig fixieren)
2. Kompass (echtes Gerät oder Handy-App) direkt daneben halten, echt Norden bestimmen
3. Sensorgehäuse so drehen, dass seine **"N"-Markierung** exakt nach Norden zeigt
4. Erst jetzt endgültig festschrauben/fixieren

Danach stimmen die angezeigten Richtungen direkt, ohne weiteren Software-Abgleich. Optionale Gegenprobe danach: `Status 10` → `ANALOG.A1` mit der Fahne in eine bekannte Richtung gedreht abfragen und mit der Tabelle im Skript vergleichen.

Wichtig: Eine Kompass-Gegenprobe funktioniert nur zuverlässig, wenn die Fahne fest auf ihrer Achse montiert ist und frei in ihre 16 Rastpositionen einrasten kann. Von Hand gehalten liefert der Rohwert keine eindeutigen Ergebnisse (zwei benachbarte Kontakte können gleichzeitig leicht aktiv sein) — kein Defekt, sondern normales Verhalten einer nicht fest montierten Fahne. Also erst nach dem endgültigen Festschrauben testen.

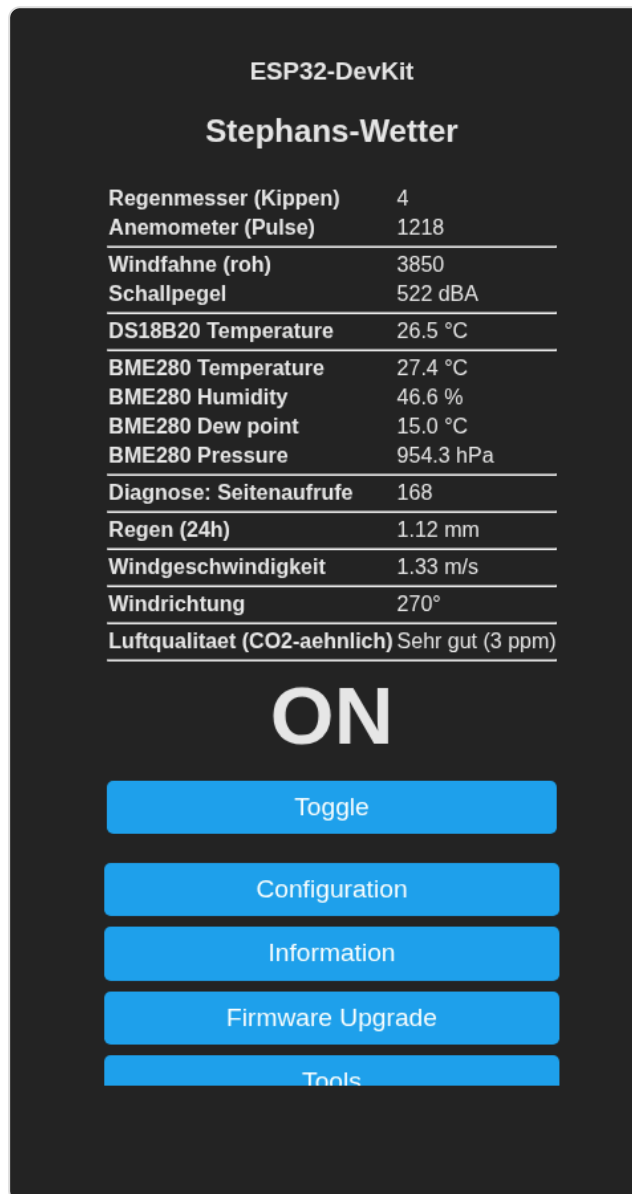
Temperatur: Falls DS18B20 oder BME280 gegenüber einem Referenzthermometer daneben liegen (z.B. durch Eigenerwärmung nahe der Elektronik), lässt sich das mit einem einzigen Befehl korrigieren:

```
TempOffset <Differenz in °C, -12.6 bis 12.6>
```

Wirkt global auf **alle** Temperatursensoren des Geräts gleichzeitig (kein Pro-Sensor-Wert in Tasmota) — am besten erst nach der finalen Montage im Gehäuse kalibrieren, da sich die Eigenerwärmung je nach Position/Gehäuse unterscheidet.

10. Fertig — Werte ablesen

Die gespeicherte Adresse `http://tasmota-EF4E84-3716.local/` (oder die feste IP aus deinem Router) im Browser öffnen — dort erscheinen alle Sensorwerte live, ganz ohne App oder Cloud-Konto. Auf dem Smartphone als Homescreen-Icon speicherbar, damit es sich wie eine App anfühlt.



Das OLED-Display zeigt die wichtigsten Kernwerte zusätzlich direkt am Gehäuse, auch ganz ohne Handy/Browser in der Nähe — vier Zeilen mit fest zugeordneter Bedeutung:

Zeile	Beispiel	Bedeutung
1	-58dBm 192.168.1.42	WLAN-Signalstärke (negativ, näher an 0 = besser; ab ca. -80 dBm wird's wacklig) + aktuelle IP-Adresse im WLAN. Zeigt stattdessen <code>No-WiFi</code> , wenn die WLAN-Verbindung gerade fehlt.

2	2M 270° 3L	M = Windgeschwindigkeit in Metern pro Sekunde (m/s). ° = Windrichtung in Grad, wie am Kompass (0°/360°=Nord, 90°=Ost, 180°=Süd, 270°=West), im Uhrzeigersinn. L = Regenmenge der letzten 24 Stunden in Liter pro Quadratmeter — das ist dieselbe Einheit wie „mm Niederschlag“ (1 mm Regen = 1 L/m²), nur anders benannt.
3	21.4C 1013 U 55%	Lufttemperatur in °C (vom wasserdichten DS18B20-Fühler, nicht vom BME280 — der sitzt zu nah an der Elektronik und würde sich selbst aufheizen), Luftdruck in hPa (ganzzahlig gerundet), Drucktrend der letzten Stunden (Up = steigt, Down = fällt, - = stabil — grober Hinweis auf Wetteränderung), Luftfeuchte in % (vom BME280).
4	Regen Stunde: 0.2L	Regenmenge der aktuell laufenden Stunde, gleiche Einheit wie Zeile 2 (L = L/m² = mm). Setzt sich zu Beginn jeder neuen Stunde automatisch auf 0 zurück.

Nachts (22:00–08:00 Uhr) bleiben Display und Status-LED bewusst aus, um nicht zu stören — das ist kein Fehler, siehe Abschnitt 7. Die Web-Oberfläche liefert dieselben Werte plus einige zusätzliche (z.B. Luftqualität, Seitenaufrufe-Zähler) jederzeit, auch nachts.

11. Troubleshooting

Symptom	Wahrscheinliche Ursache
I2CScan findet BME280/OLED nicht	SDA/SCL vertauscht, falsche I2C-Adresse, oder Board ohne sauberen Neustart (I2C-Bus-Hang — Restart 1 behebt es)
DS18B20 liefert keinen Wert	4,7 kΩ Pull-up fehlt zwischen DATA und 3V3
A1 (Windfahne) erscheint nicht in Status 10	Verkabelung des Spannungsteilers prüfen (10 kΩ zwischen 3V3 und IO34)
A1 schwankt ständig, obwohl Fahne stillsteht	Wackelkontakt am Spannungsteiler-Knoten — Widerstand-Beinchen und Windfahnen-Ader nachdrücken (siehe 5.2)
"Schallpegel"-Zeile im Web-UI zeigt Werte, obwohl kein Schallsensor verbaut ist	Bekannter kosmetischer Fallstrick — das ist der MQ135-Rohwert unter falschem, fest einkompiliertem Label (nur per Firmware-Neubau behebbar). Maßgeblich ist die Zeile "Luftqualität".
Counter1/2 zählen wild hoch ohne Bewegung	Kontaktprellen — CounterDebounce ist bereits auf 10 gesetzt, bei Bedarf höher
OLED bleibt dunkel	Verkabelung IO21/IO22/3V3/GND prüfen, I2C-Adresse ggf. 0x3D statt 0x3C (siehe 5.5)
Luftqualität zeigt dauerhaft "Kalibriere.."	Braucht 180s ununterbrochene Laufzeit nach dem letzten Neustart (siehe 5.7) — kein neuer Neustart währenddessen
Status-LED bleibt aus	Prüfen ob gerade Nachtruhe (22:00–08:00 Uhr) aktiv ist — das ist so gewollt
Status-LED leuchtet dauerhaft statt zu blinken	GPIO2 prüfen — muss Rolle "Led" (288) sein, nicht "LedLink" (544), siehe Abschnitt 7

Board komplett dunkel (LED+OLED+alles aus)	USB-C-Schnelllade-Netzteil liefert keine Spannung ohne Aushandlung — einfaches Netzteil verwenden
WLAN verbindet nicht nach dem Umtragen	SSID/Passwort nochmal per <code>Backlog SSID1 ...;Password1 ...</code> prüfen, Groß-/Kleinschreibung beachten

Viel Erfolg beim Aufbau! Bei Fragen einfach melden — die komplette, laufend aktualisierte Projekt-Doku (Originalgerät mit mehr Sensoren) steht dauerhaft unter github.com/icepaule/IceWeatherstation.